OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 2

Name: S Svara Haasini
Roll No: 2520090170

Task 1: File Copy Using System Calls (open, read, write, close)

Question:

Develop a C program using the system calls open(), read(), write(), and close() to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Source Code

haasini@SSH: ~/OSSP_upload

```
haasini@SSH:~/OSSP_upload/Week3$ cat File1.txt
Operating Systems and System Programming
This is the original content of File1.txt.
The program will copy this text into File2.txt.

haasini@SSH:~/OSSP_upload/Week3$ cat file_copy.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(void)
{
    int source_fd;
    int destination_fd;

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    /*
     * Open File1.txt in read-only mode.
     */
    source_fd = open("File1.txt", O_RDONLY);

    if (source_fd == -1)
    {
        perror("Unable to open File1.txt");
        return EXIT_FAILURE;
    }

    /*
     * Open File2.txt for writing.
     *
     * O_WRONLY  -> open for writing
     * O_CREAT   -> create it if it does not exist
     * O_TRUNC   -> erase old contents if it exists
     *
     * 0644 gives:
     * owner       -> read and write
     * group       -> read
     * others      -> read
```

```
destination_fd = open(
    "File2.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (destination_fd == -1)
{
    perror("Unable to open File2.txt");
    close(source_fd);
    return EXIT_FAILURE;
}

/*
 * Read data from File1.txt repeatedly.
 *
 * read() returns:
 * greater than 0 -> number of bytes read
 * 0              -> end of file
 * -1             -> error
 */
while ((bytes_read =
            read(source_fd, buffer, sizeof(buffer))) > 0)
{
    /*
     * Write the bytes that were read into File2.txt.
     */
    ssize_t bytes_written =
        write(destination_fd, buffer, bytes_read);

    if (bytes_written != bytes_read)
    {
        perror("Unable to write to File2.txt");

        close(source_fd);
        close(destination_fd);

        return EXIT_FAILURE;
    }
}

if (bytes_read == -1)
{
    perror("Unable to read File1.txt");

    close(source_fd);
    close(destination_fd);
```

Compilation and Execution

```
    {
        perror("Unable to read File1.txt");

        close(source_fd);
        close(destination_fd);

        return EXIT_FAILURE;
    }

    /*
     * Close both files after copying.
     */
    close(source_fd);
    close(destination_fd);

    printf("File copied successfully from File1.txt to File2.txt.\n");

    return EXIT_SUCCESS;
}
haasini@SSH:~/OSSP_upload/Week3$ gcc -Wall -Wextra file_copy.c -o file_copy
haasini@SSH:~/OSSP_upload/Week3$ echo "Compilation Successful"
Compilation Successful
haasini@SSH:~/OSSP_upload/Week3$ ./file_copy
File copied successfully from File1.txt to File2.txt.
haasini@SSH:~/OSSP_upload/Week3$ echo "----- File1.txt -----"
----- File1.txt -----
haasini@SSH:~/OSSP_upload/Week3$ cat File1.txt
Operating Systems and System Programming
This is the original content of File1.txt.
The program will copy this text into File2.txt.

haasini@SSH:~/OSSP_upload/Week3$ echo

haasini@SSH:~/OSSP_upload/Week3$ echo "----- File2.txt -----"
----- File2.txt -----
haasini@SSH:~/OSSP_upload/Week3$ cat File2.txt
Operating Systems and System Programming
This is the original content of File1.txt.
The program will copy this text into File2.txt.

haasini@SSH:~/OSSP_upload/Week3$
haasini@SSH:~/OSSP_upload/Week3$ diff File1.txt File2.txt
haasini@SSH:~/OSSP_upload/Week3$ echo "Exit status = $?"
Exit status = 0
haasini@SSH:~/OSSP_upload/Week3$ rm -f main_loop file_copy
haasini@SSH:~/OSSP_upload/Week3$
```

Observations:

• The program uses open() with flags O_RDONLY to open File1.txt for reading and O_WRONLY|O_CREAT|O_TRUNC to create File2.txt for writing (0644 permissions).

• The read() and write() system calls transfer data in 1024-byte chunks, with proper error handling for failed read/write operations and file descriptor validation.

• Compilation was successful with gcc compiler flags -Wall -Wextra for strict error checking.

• Execution output shows 'File copied successfully from File1.txt to File2.txt', confirming the program properly executed all system calls.

• The diff utility confirms File1.txt and File2.txt have identical contents (exit status 0), verifying successful file copying at the byte level.

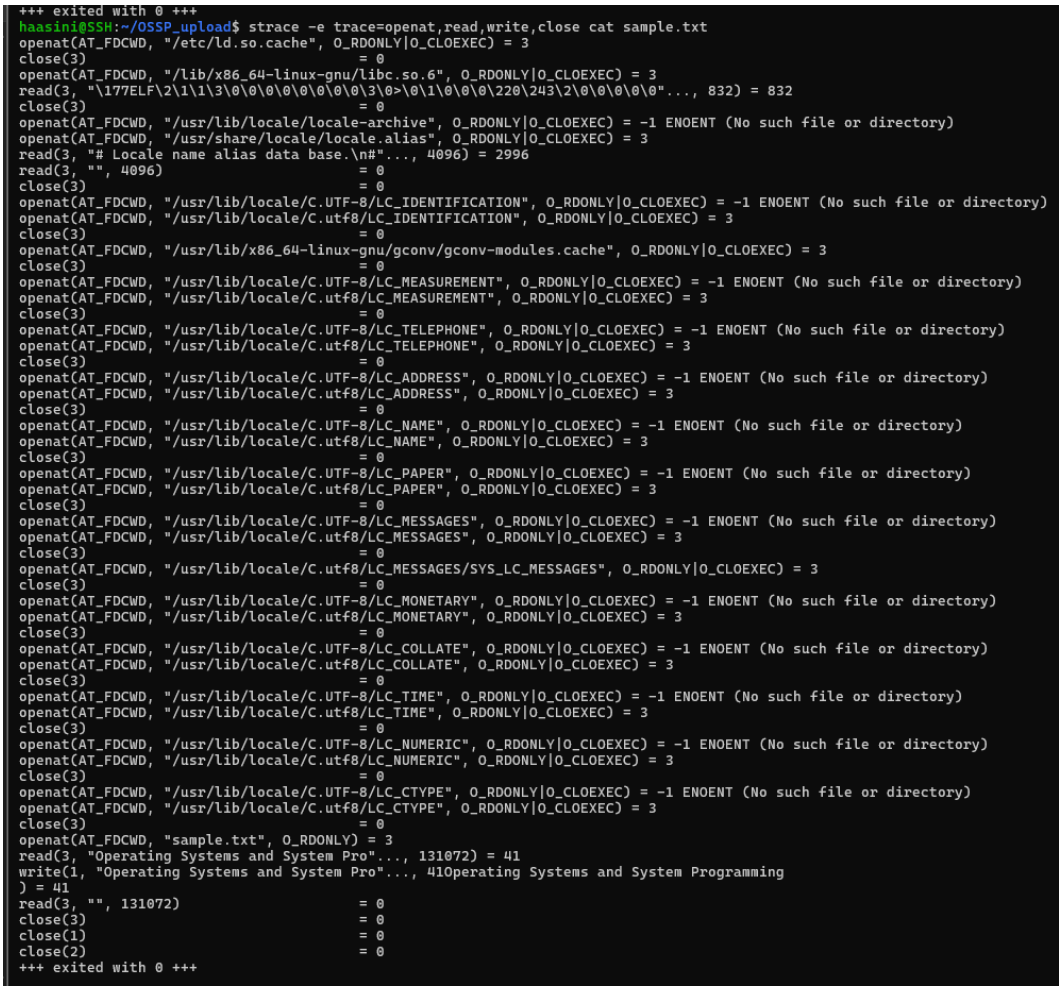
• The program demonstrates proper resource cleanup with close() calls for both file descriptors, preventing file descriptor leaks.

Task 2: System Call Tracing with strace

Question:

Use the strace utility to trace all system calls generated by the command 'cat sample.txt'. Analyze the sequence of system calls and identify the kernel services involved.

Output



```
+++ exited with 0 +++
haasini@SSH:~/OSSP_upload$ strace -e trace=openat,read,write,close cat sample.txt
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/share/locale/locale.alias", O_RDONLY|O_CLOEXEC) = 3
read(3, "# Locale name alias data base.\n#"..., 4096) = 2996
read(3, "", 4096)                       = 0
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_IDENTIFICATION", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_MEASUREMENT", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_TELEPHONE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_TELEPHONE", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_ADDRESS", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_ADDRESS", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_NAME", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_NAME", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_PAPER", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_PAPER", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_MESSAGES", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_MESSAGES", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_MESSAGES/SYS_LC_MESSAGES", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_MONETARY", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_MONETARY", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_COLLATE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_COLLATE", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_TIME", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_TIME", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_NUMERIC", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/locale/C.UTF-8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/C.utf8/LC_CTYPE", O_RDONLY|O_CLOEXEC) = 3
close(3)                                = 0
openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3
read(3, "Operating Systems and System Pro"..., 131072) = 41
write(1, "Operating Systems and System Pro"..., 41Operating Systems and System Programming
) = 41
read(3, "", 131072)                     = 0
close(3)                                = 0
close(1)                                = 0
close(2)                                = 0
+++ exited with 0 +++
```

Observations:

• The strace command with filter '-e trace=openat,read,write,close' selectively traces only file-related system calls, reducing output clutter and focusing on I/O operations.

• Multiple openat() calls appear initially for loading shared libraries and locale-related files (libc.so.6, locale-archive, UTF-8 definitions), demonstrating dynamic library loading by the OS.

• The critical sequence begins with openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3, which opens sample.txt in read-only mode and assigns file descriptor 3.

• The first read(3, ..., 131072) = 41 call reads 41 bytes from the file into a 131KB buffer, transferring data from kernel space (file system) to user space (process buffer).

• write(1, "Operating Systems and System Programming", 41) sends the 41-byte content to standard output (file descriptor 1, mapped to the terminal), demonstrating data transfer back to

kernel space.
• The subsequent read(3, "", 131072) = 0 returns 0 bytes, signaling end-of-file (EOF) and terminating the read loop in the cat program.
• close(3) = 0 successfully closes the file descriptor, releasing kernel resources associated with the open file and demonstrating proper cleanup.
• The exit status '0' at the end confirms successful execution with no errors, showing the kernel's orderly termination of the process.